

Theory

1) JavaScript has 7 primitive data types.

- * String
- * Number
- * Boolean
- * Undefined
- * Null
- * Symbol
- * BigInt

2) Primitive Type: Stores the actual value directly and is compared by value.

Reference Type: Stores the memory reference (address) of an object and is compared by reference.

3) undefined: A variable has been declared but not assigned value.

null: Represents the intentional absence of a value.

4) typeof null returns "object".

It is a historical javascript bug retained for backward compatibility.

5) Symbol is used to create unique and immutable identifiers mainly for unique object property keys.

6) BigInt is used to represent integers larger than Number.MAX_SAFE_INTEGER without losing precision.

7) string are immutable.

8) String Primitive: Primitive data type (`typeof` returns "string")
String Object: Object created using `new String()` (`typeof` returns "object").

9) Boolean values are true and false.

`Boolean(0)` → false

`Boolean("")` → false

`Boolean(null)` → false.

10) NaN stands for Not a number.
`typeof NaN` returns "number".

Output prediction:

11) string

12) number

13) undefined

14) object

15) number

16) symbol

17) BigInt

18) 53 (string)

19) 53 (string)

20) false

Error Finding

21) O/P: Object

`new String("John")` creates a string object, not a string primitive.

22) O/P: 9007199254740993

Javascript's Number type uses the IEEE 754 double precision floating pointing format.

The largest integer that can be represent exactly is

Number.MAX_SAFE_INTEGER

O/P: 9007199254740993

23) TypeError: Cannot mix BigInt and other types

10n → BigInt

10 → Number

Javascript does not automatically convert between BigInt and number because doing so could lose precision.

Therefore arithmetic between them is not allowed

24) O/P: True

= is the loose equality operator.

It allows type correction.

25) O/P: False

=== is the strict equality operator

It checks 1) value

2) data type

26) O/P: hello

String in Javascript are immutable.

That means their individual characters cannot be modified after the string is created.

s[0] = "H";

is simply ignored.

No error is thrown, but nothing changes.

27) O/p: true

type of NaN

"number"

"number" === "number"

NaN stand for not a number, yet javascript still classifies it as the number type

28) O/p: false

Every ~~new~~ call to symbol() creates a new unique Symbol.

29) O/p: true

== performs type coercion.

30) False

=== check both value and type.

Comparison

0 == false

Type Conversion

Yes

O/p

true

0 === true

No

false

Coding

31)

// string

let name = "John";

// Number

let age = 25;

// Boolean

let isStudent = true;

// undefined

let address;

// Null

let phone = null;

// symbol

let id = Symbol("user");

// BigInt

let salary = 9007199254740993n;

console.log(typeof name); // string

console.log(typeof age); // number

console.log(typeof isStudent); // boolean

console.log(typeof address); // undefined

console.log(typeof phone); // null

console.log(typeof id); // symbol

console.log(typeof salary); // big int

32) let message = "hello";

message[0] = "H";

console.log(message);

O/P: hello

String are immutable. Changing a character using its index has no effect.

Correct Ans:

message = "Hello";

console.log(message);

O/P "Hello"

```

33) let num1 = 100n;
    let num2 = 50n;

    let result = num1 + num2;

    console.log(result);

```

O/p: 150n

Both operands are BigInt.

```

34) function isNullOrUndefined (value) {
    return value value === null || value === undefined;
}

```

```

console.log(isNullOrUndefined(null));
console.log(isNullOrUndefined(undefined));
console.log(isNullOrUndefined(10));
console.log(isNullOrUndefined("Hello"));

```

O/p: true
true
false
false

```

35) let sym1 = Symbol("id");
    let sym2 = Symbol("id");
    console.log(sym1 === sym2);

```

O/p: false.

```

36) console.log(Boolean(1));
    console.log(Boolean(0));
    console.log(Boolean(""));
    console.log(Boolean("JavaScript"));
    console.log(Boolean(null));

```

O/p: true
false
false
true
false

37) let value = NaN;
console.log(value === value);
console.log(Number.isNaN(value));

O/P true
true

38) let a = "5";
let b = 5;
console.log(typeof a);
console.log(typeof b);

O/P:
string
number

39) let value 1;
let value 2 = null;
console.log("Undefined value:", value 1);
console.log("Null value:", value 2);

O/P undefined value: undefined
Null value: null

40) let age = 22;
let isStudent = true;
console.log('Age: \${age}, student: \${isStudent}');

O/P Age: 22, student: true.

Real-world scenario:

41) symbol

A symbol creates unique value every time it is

Created, even if two symbols have the same description.

Eg):

```
const userId = Symbol("id");  
const productId = Symbol("id");  
console.log(userId === productId);
```

O/p
false

42) null

null indicates that the value is intentionally empty or intentionally unknown.

undefined usually means the variable has not been assigned any value at all.

43) Use BigInt

The number type cannot safely represent very large integers because it has a maximum safe integer limit.

Eg):

```
const amount = 99999999999n;  
console.log(amount);
```

O/p 99999999999n

44) use Boolean

A boolean can represent only two states.

- true

- false

Eg)

```
let isLoggedIn = true;  
if (isLoggedIn) {  
  console.log("welcome!");  
}
```


else {

console.log("Please log in.");

45)

let username = "mala"

console.log(typeof username === "string")

O/P: true

46)

let quantity = 0;

if (!quantity) {

console.log("No quantity");

}

O/P: No quantity

A quantity of a 0 is a valid value. but it is false, so the condition incorrectly treats it as if the value were missing.

it may be a deliberately

47)

store it as a string

Phone numbers are identifiers not numbers used in mathematical calculations. if stored as a number.

let phone = 0987654321;

the leading zero is not preserved.

store it as

let phone = "0987654321";

48)

Javascript uses IEEE 754 floating point arithmetic so decimal values cannot always be represented exactly

Compare numbers using a tolerance :

$\text{Math.abs}(0.1 + 0.2) - 0.3 < \text{Number.EPSILON}$

For financial applications many systems also store monetary values as the smallest currency unit using integers to avoid floating-point precision issues.

49) undefined

```
let value;
```

```
if (value === undefined) {
```

```
  console.log("variable is missing");
```

```
}
```

For a variable intentionally set to null:

```
let value = null;
```

```
if (value === null) {
```

```
  console.log("value intentionally empty");
```

```
}
```

undefined - no value has been assigned.
null → Intentionally assigned "no value".

50)

type of null returns "Object"

Eg) `console.log(typeof null);`

O/p: Object

Correct way:

```
let value = null;
```

```
if (value === null) {  
    console.log("value is null");  
}
```

The expression `value === null` accurately checks whether the value is null, whereas `typeof` ~~as~~ cannot distinguish null correctly.